

VIRTUAL LAB PLATFORM

Tech Stack Decision First

Before anything else, some adjustments to your proposed stack worth considering.

Frontend: React + Vite is perfect. Keep it.

Styling: Material UI is fine for dashboard UI but for the simulation canvas and electronic components, you will be fighting MUI constantly. Use MUI for the application shell, sidebar, panels, and modals. Use plain CSS or Tailwind for the simulator canvas itself.

Canvas Rendering: This is the most critical decision. Use Konva.js which is a React-friendly canvas library. It handles drag and drop, hit detection, zooming, panning, and layering out of the box. Raw HTML5 Canvas or SVG will become unmaintainable fast. Konva sits on top of canvas and gives you a scene graph.

Circuit Simulation Engine: This runs entirely in the browser. Use math.js for matrix operations needed by the nodal analysis solver. No backend involvement here.

Graphs and Oscilloscope: Use uPlot. It is the fastest time-series plotting library available for JavaScript, handles real-time streaming data, and is far lighter than Chart.js or Recharts for this use case.

Code Execution for Wokwi-like IoT: Use your existing Piston API bridge through Django. Keep this.

Backend: Django REST Framework is good. Keep it.

Database: PostgreSQL is only needed for user accounts, saved circuits, project history, and sharing. Not needed for simulation itself.

WebSockets: Django Channels for real-time collaboration and live simulation streaming between browser and backend if needed later.

Additional suggestion: AVR8js for Arduino microcontroller simulation in the browser. This is exactly what Wokwi uses internally. It interprets compiled AVR bytecode and simulates the chip registers, timers, and GPIO pins. Pair it with your circuit solver and you get genuine IoT simulation.

Phase 1: Project Setup and Application Shell

Step 1: Initialize the project

Create your Vite React project. Set up folder structure immediately before writing any components because reorganizing later is painful.

Your folder structure should be:

```
src/  
  core/  
    solver/      ← circuit simulation engine lives here  
    components/  ← electronic component definitions  
    models/      ← component electrical models  
  canvas/  
    SimulatorCanvas/ ← Konva canvas wrapper  
    Grid/         ← background grid  
    Wire/         ← wire drawing and routing  
    Node/         ← connection points  
  instruments/  
    Oscilloscope/  
    Multimeter/  
    FunctionGenerator/  
    PowerSupply/  
  components-library/  
    passive/      ← resistor, capacitor, inductor  
    active/       ← op-amp, transistor, diode  
    iot/          ← Arduino, ESP32, sensors  
    sources/      ← voltage source, current source  
    gates/        ← AND, OR, NOT etc  
  panels/  
    ComponentPanel/ ← left sidebar drag source  
    PropertiesPanel/ ← right sidebar component settings  
    InstrumentPanel/ ← bottom instrument rack  
  ui/             ← MUI application shell components  
  store/          ← Zustand state management  
  api/            ← Django REST client
```

Step 2: State management

Use Zustand instead of Redux. It is dramatically simpler and fast enough for this use case. You need one central store managing the circuit state — which components exist, where they are placed, how they are connected, current simulation values at each node.

Your circuit state shape should look like this conceptually: a components map keyed by unique ID containing position, type, and property values. A wires array containing pairs of connection point IDs. A nodes map tracking which connection points share electrical nodes after wire connections are resolved. A simulation state containing current voltage and current values per node updated every simulation tick.

Step 3: Install core dependencies

Konva and react-konva for canvas. math.js for the solver matrix operations. uPlot for oscilloscope graphs. Zustand for state. uuid for component IDs. Django Channels on the backend side.

Phase 2: The Simulation Canvas

This is the heart of the application. Get this right before building anything else.

Step 4: Konva canvas setup

Your SimulatorCanvas component wraps a Konva Stage. The Stage contains multiple Layers in this order from bottom to top: Grid Layer for the background dot or line grid, Wire Layer for drawn connections, Component Layer for draggable components, Selection Layer for selection rectangles and highlights, Instrument Overlay Layer for oscilloscope probes.

The Stage handles zoom with mouse wheel and pan with middle mouse button or space+drag. These are non-negotiable for usability. A circuit that you cannot zoom and pan is immediately frustrating.

Step 5: The grid system

Draw a dot grid using Konva. Grid spacing should be 20px at 100% zoom. All component placement and wire routing snaps to this 20px grid. This is critical — without snapping, wires never align to pins and the circuit looks messy. Every position in your system should be stored in grid units not pixels, and converted to pixels only for rendering.

When zooming, grid lines subdivide. At 200% zoom show 10px subdivisions. At 50% zoom consolidate to 40px spacing. This is standard behavior users expect.

Step 6: Wire drawing system

This is complex and worth getting right early. When a user clicks a component pin, they enter wire drawing mode. As they move the mouse, you draw an orthogonal wire — only horizontal and vertical segments, never diagonal. This is standard in all professional EDA tools.

The routing algorithm finds a path between two points using only right-angle turns. A simple approach is Manhattan routing — go horizontal first then vertical, or vertical first then

horizontal, choosing whichever avoids known component bounding boxes. Store wires as arrays of waypoints. Allow users to drag wire segments to reroute them.

Wires that share an electrical node show a junction dot where they meet. Wires that cross without a junction are not connected. This visual grammar is fundamental to circuit readability.

Step 7: Component pin system

Every component has defined pin positions relative to its origin in grid coordinates. A resistor has two pins, one at each end. An op-amp has five pins — two inputs, one output, two supply. When you rotate a component, pin positions rotate accordingly.

Pins are invisible small circles that become visible on hover, glowing to invite connection. When a wire endpoint is dragged near a pin, it snaps to it. A connected pin shows a filled dot, an unconnected pin shows an open circle.

Phase 3: Component Library

Step 8: Component definition system

Every electronic component is defined by three things: its visual representation, its pin layout, and its electrical model for the solver.

Create a component definition registry. Each entry specifies the component type string as a key, then contains the SVG or Konva drawing instructions for rendering, the pin positions and their roles like positive negative input output, the default property values like resistance value in ohms, the property schema for the properties panel like min max step units, and the spice-like model parameters for the solver.

Step 9: Passive components to build first

Resistor: Two-terminal, value in ohms. Draw as a rectangle or zigzag line. This is your simplest component and your solver test case.

Capacitor: Two-terminal, value in farads. Draw as two parallel lines. Has time-dependent behavior in transient simulation.

Inductor: Two-terminal, value in henries. Draw as a series of bumps. Also time-dependent.

Ground: Single terminal reference node. Every circuit needs at least one. This is the zero-volt reference.

Voltage source: Two-terminal. Can be DC, sinusoidal, square wave, or arbitrary. Has amplitude, frequency, offset, and phase properties.

Current source: Two-terminal. Same waveform options as voltage source.

Step 10: Semiconductor components

Diode: Two-terminal with anode and cathode. Model using the Shockley diode equation. Properties include forward voltage and ideality factor.

BJT Transistor: Three-terminal NPN or PNP. Use the Ebers-Moll model for realistic behavior. Properties include beta, saturation current.

MOSFET: Three-terminal N or P channel. Properties include threshold voltage, transconductance.

Step 11: Op-amp and analog ICs

Generic op-amp: Five-terminal. Model as voltage-controlled voltage source with very high open-loop gain, finite bandwidth, output current limit. This single model covers inverting amplifier, non-inverting amplifier, integrator, differentiator, comparator, Schmitt trigger — all just by changing the external component connections.

Specific ICs like LM741, LM358, LM393 comparator, NE555 timer. Each has a behavioral model capturing its datasheet characteristics.

Step 12: Digital logic gates

AND, OR, NOT, NAND, NOR, XOR, XNOR. Model as ideal switches for digital simulation. These are simpler because you are not solving analog node voltages — you are propagating logic levels with defined propagation delays.

Seven-segment display as an output component. LED array. These have no electrical model complexity — they just read their input node voltage and render accordingly.

Step 13: IoT components

Arduino Uno and Nano: This is where AVR8js comes in. The component renders as the board image. It has digital pins, analog pins, power pins, and ground pins — all as connectable terminals. When simulation runs, AVR8js executes the user's compiled sketch and the GPIO register states drive the pin voltage outputs into your circuit solver.

ESP32: Similar approach using an ESP32 emulator. More complex because of WiFi simulation but for basic GPIO and ADC purposes the same pattern works.

Sensors: DHT11 temperature and humidity, HC-SR04 ultrasonic distance, PIR motion, LDR light dependent resistor. These are behavioral models — given a simulated environment input parameter the user can set, they output the appropriate signal on their data pins.

Servo motor: Input is PWM signal, output is a visual angle indicator showing the shaft position.

LCD display: 16x2 character display that actually renders text based on the I2C or parallel data coming from the microcontroller simulation.

Phase 4: The Electrical Solver

Step 14: Netlist extraction

Before solving anything, you need to extract a netlist from the visual circuit. Walk all components and their connections, assign sequential integer IDs to each unique electrical

node where ground is always node zero, and build a list of elements each specifying type, connected nodes, and value.

This netlist is the input to your solver and is completely independent of the visual representation. Regenerate it every time the circuit changes.

Step 15: DC operating point solver

Implement Modified Nodal Analysis. You build a conductance matrix G and a current vector I , then solve G times V equals I for the node voltage vector V .

For a resistor between nodes A and B with resistance R : add $1/R$ to $G[A][A]$, add $1/R$ to $G[B][B]$, subtract $1/R$ from $G[A][B]$ and $G[B][A]$.

For a voltage source between nodes A and B with value V : add a new unknown current variable, extend the matrix accordingly using the standard MNA stamp for voltage sources.

Solve the linear system using math.js LU decomposition. The result gives you voltage at every node. Current through every element is then computed from the voltage differences and element values.

For nonlinear elements like diodes and transistors, use Newton-Raphson iteration. Linearize the nonlinear element around an operating point, solve the linear system, update the operating point, repeat until convergence. Typically converges in 5 to 15 iterations.

Step 16: Transient simulation

Step through time. At each timestep, capacitors become companion model equivalents — a current source in parallel with a resistor, with values depending on the capacitor voltage from the previous timestep and the timestep size. Inductors similarly become voltage sources in series with resistors.

Use backward Euler for stability or the trapezoidal method for accuracy. Timestep can be adaptive — when signals are changing rapidly, use smaller steps. When signals are stable, use larger steps. This is how Spice handles it.

Run the simulation loop in a Web Worker so it does not block the main thread and freeze your UI. Post simulation results back to the main thread via message passing. The main thread updates the visual component states and feeds data to the oscilloscope.

Step 17: Frequency domain analysis

For AC analysis, replace component models with their complex impedance equivalents — a capacitor has impedance 1 divided by $j \omega C$, an inductor has impedance $j \omega L$. Sources become phasors. Solve the complex linear system at each frequency point. This gives you magnitude and phase at every node as a function of frequency. Plot it and you have a Bode plot generated directly from the student's circuit.

Phase 5: Instruments

Step 18: Oscilloscope

This is a full instrument widget, not just a graph. Build it as a panel that can be docked at the bottom or floated as a window.

It has a screen area rendered with uPlot with a dark background and a grid overlay. It has channel controls — CH1 and CH2 each with volts per division knob, position knob, and coupling selector for AC DC or ground. It has timebase control — time per division. It has trigger controls — source, level, edge, and mode for auto normal and single.

To use it, students drop probe markers onto circuit nodes in the canvas. The probe markers are Konva elements that sit on specific nodes and continuously read the simulation voltage values. These values stream into the oscilloscope ring buffer and the display updates in real time.

The trigger system holds the waveform stable. When the signal crosses the trigger level on the specified edge, the oscilloscope starts a new sweep from that point. This is what makes a periodic waveform appear stationary on screen.

Add cursor measurements — two vertical cursor lines the user can drag, with readout showing the time difference and frequency. Add automatic measurements — the oscilloscope computes and displays frequency, period, amplitude, RMS, rise time, and duty cycle automatically.

Step 19: Function generator

Produces a test signal. Waveform shape selector for sine, square, triangle, sawtooth. Frequency control from 1 Hz to 100 MHz. Amplitude and offset controls. Duty cycle for square wave. Phase control.

This connects to a voltage source component in the circuit. When the student connects the function generator output to a circuit node, that node is driven by the generated waveform feeding into the transient simulation.

Step 20: DC power supply

Two or three channels each with voltage and current limit controls. Adjustable from zero to 30 volts. Current displays actual drawn current computed by the solver. Goes into overvoltage or overcurrent protection state visually if limits are exceeded.

Step 21: Multimeter

Probe-based just like the oscilloscope. Modes for voltage DC and AC, current DC and AC, resistance, capacitance, diode test, and continuity. In resistance mode it injects a small test current and measures the resulting voltage drop. In continuity mode it checks if node resistance is below a threshold and plays a beep. Displays a realistic seven-segment style readout.

Step 22: Spectrum analyzer

Takes a time-domain signal from a circuit node and computes the FFT. Displays frequency content. Students can see harmonic distortion in a clipping amplifier, or verify the cutoff frequency of a filter. X axis is frequency on a logarithmic scale, Y axis is magnitude in decibels.

Phase 6: IoT Simulation Layer

Step 23: Code editor integration

Embed Monaco Editor. Language support for Arduino C++ with syntax highlighting and autocomplete for standard Arduino functions. A compile button triggers the backend.

Step 24: Arduino compilation backend

In Django, set up an endpoint that receives Arduino sketch source code and returns compiled AVR ELF binary. Use arduino-cli on the server — it is a command line tool that can compile sketches programmatically. Return the compiled binary bytes to the frontend.

Step 25: AVR simulation in browser

Feed the compiled binary into AVR8js. This runs the AVR microcontroller cycle by cycle in a Web Worker. The simulation loop calls a tick function on the AVR emulator every simulated microsecond. After each tick, read the GPIO port registers and map them to the corresponding Arduino pin components in your circuit. These become voltage values fed into the circuit solver. Analog inputs read from the circuit solver node voltages and write into the ADC registers of the AVR emulator.

This bidirectional coupling is what makes the IoT simulation live. The Arduino code actually executes. The circuit actually responds. A digitalWrite HIGH on pin 13 actually turns on a connected LED in the circuit simulation.

Step 26: Serial monitor

The AVR emulator's UART registers can be read every tick. Characters written to Serial.print in the Arduino code appear in a serial monitor panel in real time. Users can also type into the serial monitor and those characters go into the UART receive register.

Phase 7: Properties Panel and Component Configuration

Step 27: Context-sensitive properties panel

When a user clicks a component, the right panel populates with that component's editable properties. A resistor shows a value input with unit selector for ohms kilohms megaohms. A capacitor shows value with picofarads nanofarads microfarads. A voltage source shows type selector and the relevant parameters for that type.

Changing a value instantly regenerates the netlist and triggers a new solver run. The simulation updates in real time as the user types or drags a slider.

For the Arduino component, clicking it opens the code editor in a modal or split panel.

Step 28: Component rotation and flip

Right click context menu or keyboard shortcut R to rotate 90 degrees. Keyboard shortcut F to flip horizontally. Wire connections follow the pin positions after transformation.

Phase 8: Django Backend

Step 29: Database models

User model extending Django's auth user. Circuit model with fields for owner, title, created and updated timestamps, and a JSON field storing the complete circuit state — component positions, types, values, and wire connections. This JSON is exactly your Zustand store serialized.

CircuitVersion model for history. SharedCircuit model for public sharing with a unique URL slug.

Project model grouping multiple circuits together into a lab assignment.

Step 30: API endpoints

Save circuit endpoint receiving the JSON circuit state and storing it. Load circuit endpoint returning the stored JSON. List circuits for the authenticated user. Fork circuit for copying someone else's public circuit. Compile Arduino code endpoint proxying to arduino-cli and returning binary. Export circuit as image or PDF.

Step 31: Collaboration with Django Channels

A WebSocket endpoint where multiple users can join a circuit session by ID. When one user moves a component or changes a value, the delta is broadcast to all other connected users. Their stores update and their canvases reflect the change. Use operational transforms or last-write-wins depending on complexity tolerance.

Phase 9: Polish and Advanced Features

Step 32: Simulation controls

Play, pause, and reset buttons for the simulation. Speed control — run at 1x realtime or accelerate to 100x for watching slow RC charging. Single step mode advancing one timestep at a time for debugging.

Step 33: Measurements and annotations

Let users place measurement annotations directly on the canvas — node voltage labels that display the current simulated value, current arrows showing direction and magnitude on wire segments, power dissipation indicators on resistors showing watts as a color heat map.

Step 34: Export capabilities

Export circuit schematic as SVG or PNG. Export simulation data as CSV. Export as SPICE netlist so advanced users can import into LTSpice. Export Arduino sketch as a downloadable .ino file.

Step 35: Component search and favorites

The left panel has a search box filtering the component library by name or category. Users can star favorite components for quick access. Recent components section showing last used items.

Step 36: Built-in lab experiments

Pre-built circuit templates for common experiments. Inverting amplifier with gain calculation sidebar. Wheatstone bridge. RC low-pass filter with automatic Bode plot. Half-wave and full-wave rectifier. 555 timer astable and monostable. These load as starting points that students can modify.

Step 37: Error detection

Real-time circuit validation before solving. Detect floating nodes — nodes with no DC path to ground. Detect short circuits between supply and ground. Detect components with no connections. Show warnings with clear explanations rather than letting the solver fail silently or produce nonsensical results.

Build Order Summary

Build in this exact sequence to always have something working:

First get the Konva canvas with grid, zoom, and pan working with no components yet.

Second add the component registry with just a resistor and voltage source, drag from panel, place on canvas, rotate.

Third implement pin detection and wire drawing between two pins.

Fourth build the netlist extractor from the visual canvas state.

Fifth implement the basic MNA solver for resistor networks with voltage sources and verify with hand calculations.

Sixth add the oscilloscope widget connected to node values from the solver.

Seventh add transient simulation with capacitors so you can simulate an RC charging curve.

Eighth add the op-amp model and build the inverting comparator so you can replace your existing hardcoded simulation with a real one.

Ninth add the Arduino component and AVR8js integration for IoT simulation.

Tenth add Django backend for saving and loading circuits.

Everything else — more components, more instruments, collaboration, exports — layers on top of this foundation.

This order ensures you never have a long period where nothing is visible and working. Every phase produces something demonstrable.